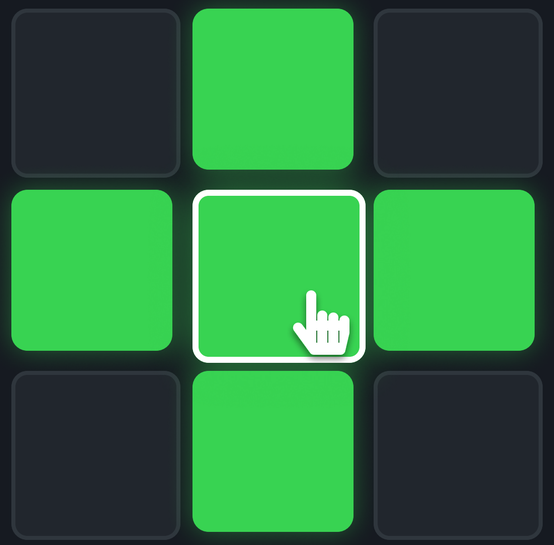
AI – Assignment I

Lights Out Puzzle Solver

BACKGROUND • PROBLEM FORMULATION • IMPLEMENTATION

Specification

Toggle Mechanics



Clicking the center cell toggles its state and its four adjacent neighbors (N, S, E, W).

Puzzle Definition

Played on an N × N grid of lights (binary states: ON/OFF)

Action Mechanic

Clicking toggles target cell + orthogonal neighbors

Objective

Find sequence of presses to turn ALL lights OFF

Complexity

State space grows exponentially ($2^{N \times N}$)

Why Search?

Manual solution difficult; algorithmic search is necessary

Background & Existing Approaches

DeepCubeA

Combines A* search guided by deep neural networks to solve complex combinatorial puzzles.

(Agostinelli et al., 2019, Nature)

Q* Search Algorithm

Specialized search algorithm designed for large action spaces, applied specifically to Lights Out state graphs.

(arXiv:2102.04518)

Heuristic Search Algorithms

Implementing an Algorithm like A*, or a BFS or DFS with a custom cost function typically leads to a faster solution in problems with a large state space

Heuristic Search

Methodology Benchmark

Compare the performance of our search against a algebraic and optimal solution

Linear Algebra vs. Search

Search Problem Formulation

State Representation

N×N binary matrix
(1 = ON, 0 = OFF)

Initial State

Randomized solvable board
(reverse-toggles from empty)

Goal Test

All lights OFF
(sum of matrix = 0)

Operators

Toggle(x, y) flips the target cell and its valid orthogonal neighbors (up, down, left, right)

EVALUATION FUNCTION

$$f(n) = g(n) + h(n)$$

Heuristic Function h(n)

h(n) = ceil(L/5), where L is the number of lights currently ON. (Divided by 5 because one move toggles at most 5 lights)

Algorithms and Heuristics

BFS
Time complexity: O(b^d)
Space complexity: O(b^d)

UCS
Time complexity: O(b^(1+[C*/ε))
Space complexity: O(b^(1+[C*/ε))

Greedy Search
Time complexity: O(b^m)
Space complexity: O(b^m)

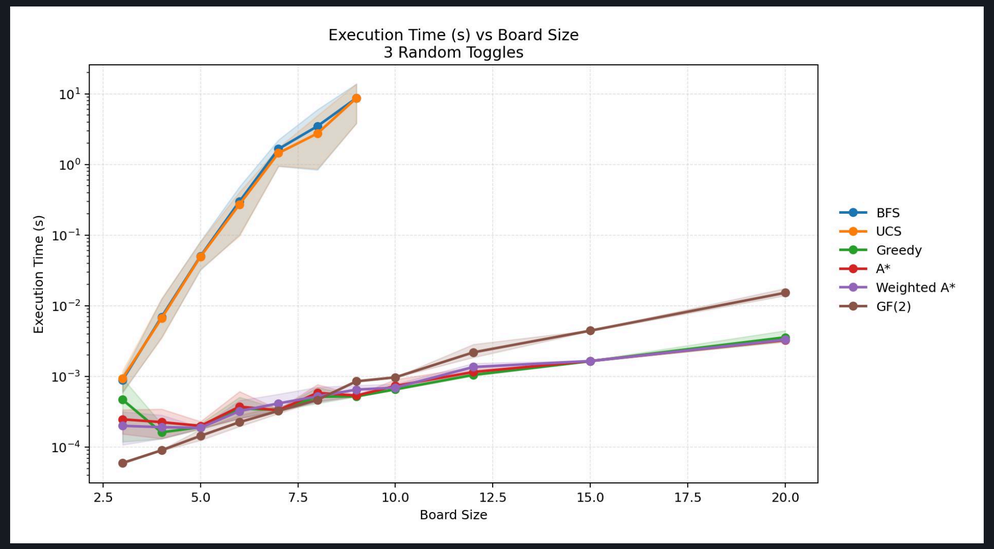
A*
Time complexity: O(b^d)
Space complexity: O(b^d)
f(n) = g(n) + h(n)

Weighted A*
Time complexity: O(b^d)
Space complexity: O(b^d)
f(n) = g(n) + 2*h(n)

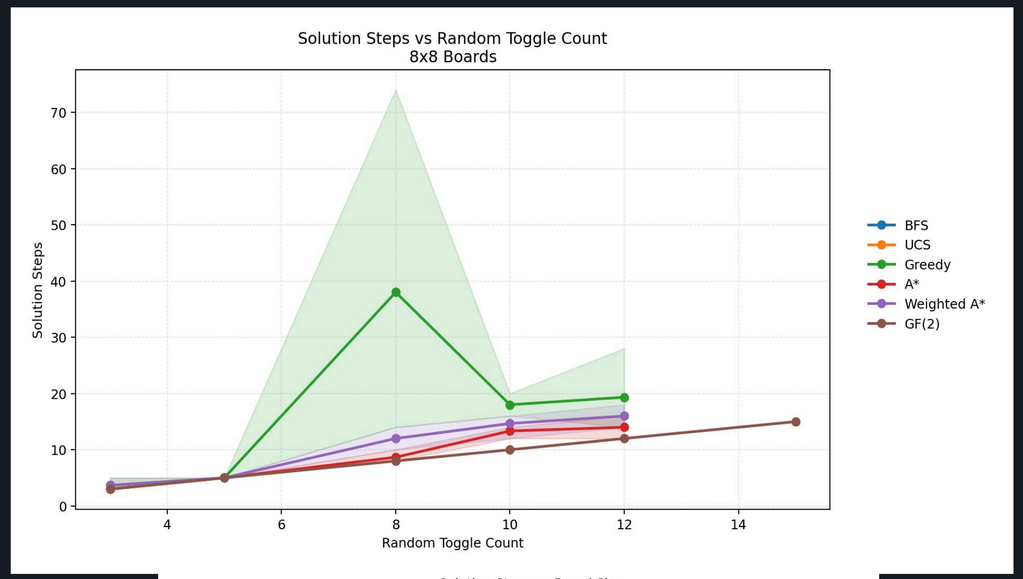
Gaussian Elimination
Time complexity: O(b^6)
Space complexity: O(b^3)

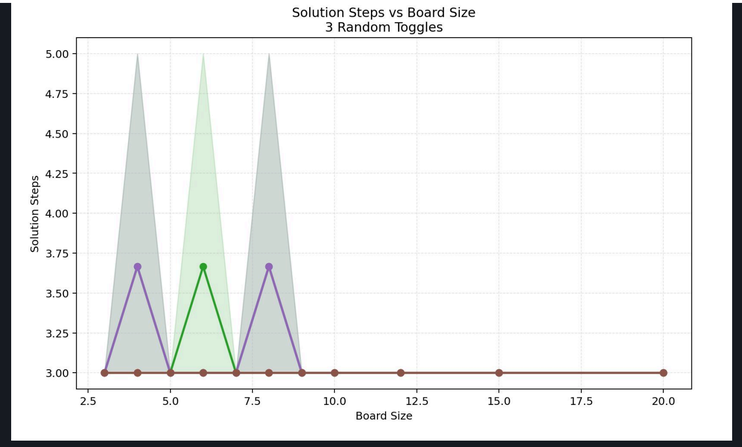
Results

Execution Time, depth of 3



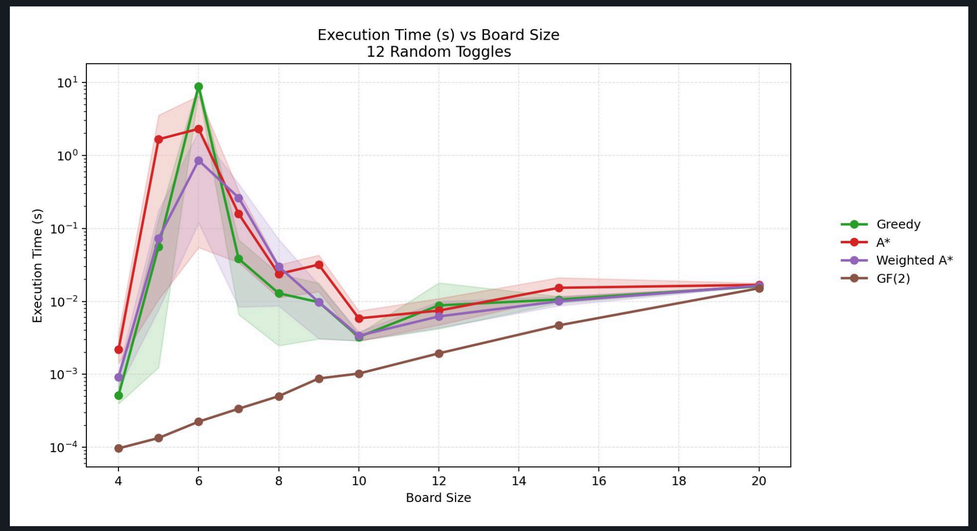
Solution found vs Optimal solution



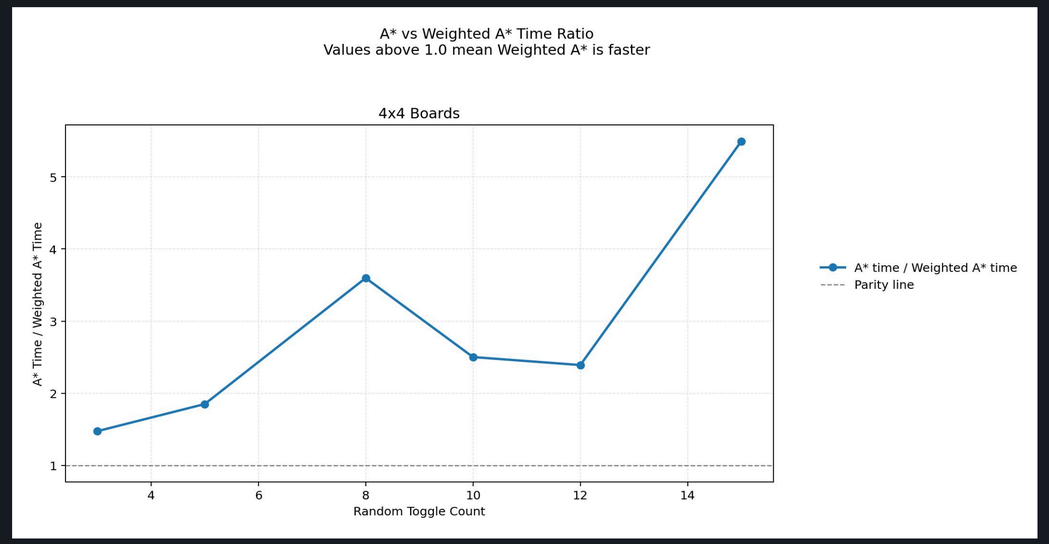


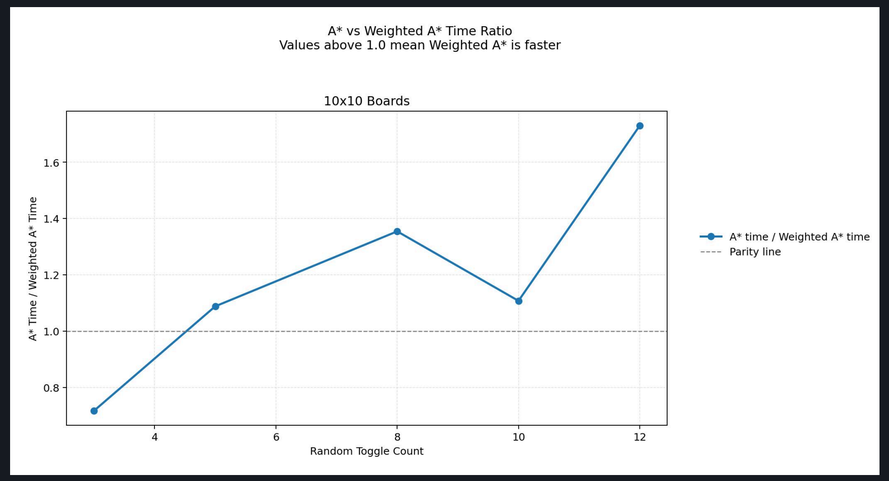
Results

The effect of a sparse board



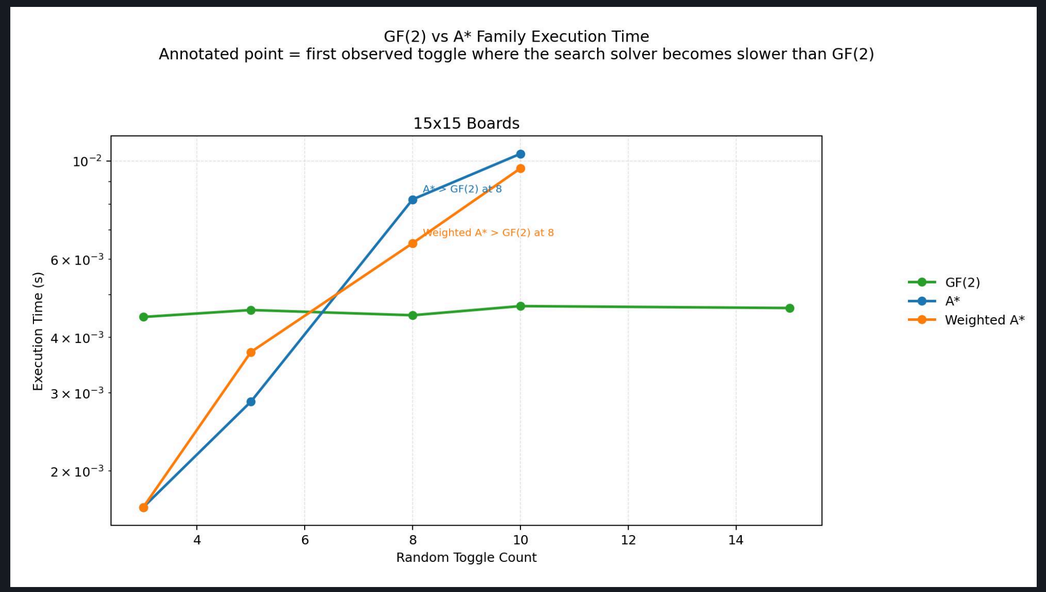
A* vs Weighted A* Ratio



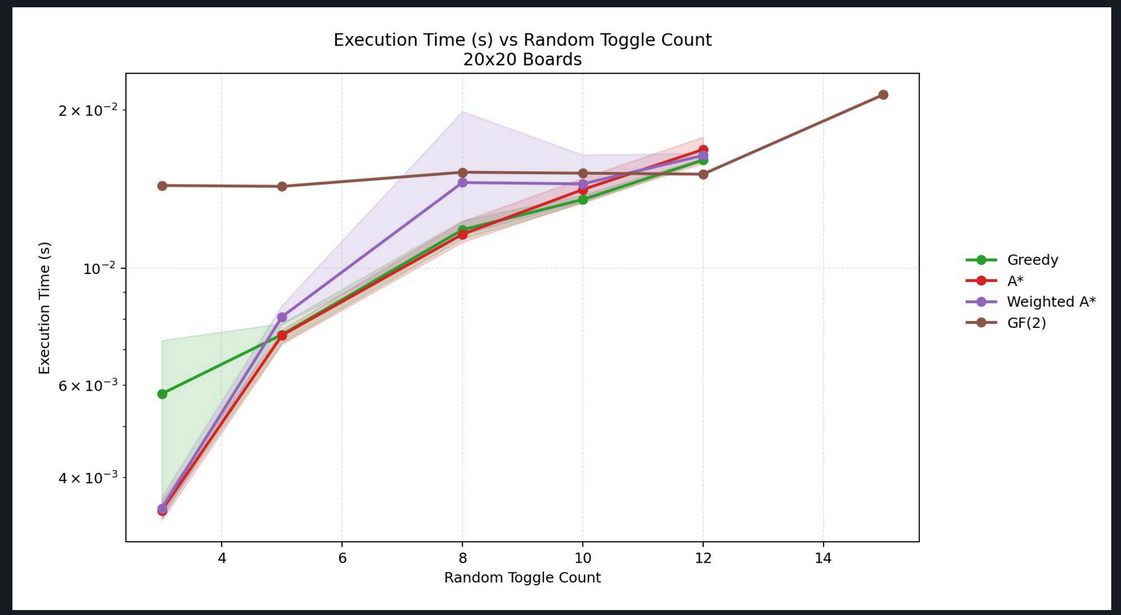


Results

Heuristic Search vs Gaussian Elimination



Execution times for 20x20



Comparison and Conclusion

BFS and UCS vs Heuristics

Even though BFS and UCS always ensure an optimal solution, algorithms like the A* and Greedy search made it possible to solve much larger and complex boards.
Most of the times, the heuristic algorithms actually found the optimal solution

A* vs Weighted A*

Our Weighted A* efficiency differed from instance to instance: sometimes faster that A* and sometimes slower.
Weighted A* displayed better performance mainly in problems with a deeper solution

Linear Algebra vs Heuristic Search:

The Linear Algebra implementation execution time is constant for puzzles with same size and different branch depth. As such, the Gaussian Elimination method is the ideal algorithm to use for more complex puzzles (more toggles) than using heuristics.

The Sparse board

Surprisingly, the execution time and number of visited states was lower for the heuristic algorithms when the board was larger. This may have happened because on a larger board it is less likely for toggles overlap and generate patterns that deviate from the heuristic.

References

- Lights Out: An Application of Linear Algebra over a Finite Field, https://people.math.ethz.ch/~acannas/Student_Papers/BSc_Theses/2022_bsc_coulibaly_lights_out.pdf
- Lights Out Wikipedia page, https://en.wikipedia.org/wiki/Lights_Out_(game)
- Lights Out Online game, https://www.lightsout.ir/